CLOUD-05: Azure Integration

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 5 of 8 | **Created:** March 2026 | **Last Updated:** 11/11/2026

Overview

This notebook covers Dynatrace's integration with Microsoft Azure: the connection options and how they authenticate, which Azure services are covered and at what depth, how to query Azure resources and Azure Monitor metrics with DQL, how to forward Azure activity, Entra ID and resource logs into Grail, how to run Dynatrace on Azure Kubernetes Service (AKS), and how Azure's subscription and resource-group structure maps onto Dynatrace governance.

Table of Contents

1. Azure Integration Architecture
2. Authentication Setup
3. Supported Azure Services
4. Querying Azure Resources
5. Azure Metrics with DQL
6. Forwarding Azure Logs and Events
7. AKS Monitoring
8. Subscription and Resource Group Governance
9. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail (the DQL cells do not run on Dynatrace Managed)
Permissions	`storage:metrics:read`, `storage:smartscape:read`, `storage:entities:read`, `storage:logs:read`, `storage:buckets:read` to run the queries; `settings:objects:read` / `settings:objects:write` to manage connections
Azure	A subscription (or management group) where you can create an Entra ID app registration and assign roles

Requirement	Details
Connection	An Azure connection in the Clouds app (recommended), the Azure Native Dynatrace Service, or the classic Azure integration
Prior Knowledge	CLOUD-01 fundamentals; CLOUD-03 for the Kubernetes material in §7

1. Azure Integration Architecture

Dynatrace offers several integration paths for Azure. They are not alternatives to one another so much as answers to different questions — *what does Azure measure about my resources*, *what do my resources log*, and *what does my code do*.

Connection Methods

Method	Signal	Mechanism	Best For
Azure connection in the Clouds app	Resource metrics, topology, metadata	Dynatrace SaaS calls Azure APIs directly with an Entra ID app registration — no ActiveGate to run	Net-new Azure onboarding on SaaS
Azure Native Dynatrace Service	Metrics (optional), activity + resource logs, OneAgent management	An Azure resource bought through Azure Marketplace; creates a system-managed identity and diagnostic settings for you	Azure-first teams who want Azure-side billing and lifecycle
Classic Azure integration	Resource metrics	Configured under Settings; an ActiveGate with the Azure module polls the Azure Monitor API every 5 minutes	Existing classic configurations; Dynatrace Managed
Diagnostic settings → Event Hubs	Activity, Entra ID and resource **logs**, plus events	Azure streams logs to regional Event Hubs; Dynatrace pulls from them (§6)	Log ingestion — this is a log path, not a metrics path
OneAgent code-level tracing	Distributed traces	OneAgent on the VM, App Service, or container auto-instruments Azure SDK calls made **by your application processes**	Seeing *which code path* called *which Azure service*, and what it cost

> **Three signal classes.** Rows 1–3 are **resource-metric** paths — they read what Azure Monitor already measures *about* an Azure resource. Row 4 is the **log** path. Row 5 is the **code-level tracing** path — your application's own calls *into* Azure services. Most Azure estates need all three; §3 draws the metric-vs-trace distinction in detail and §6 covers logs.

Azure Native Dynatrace Service

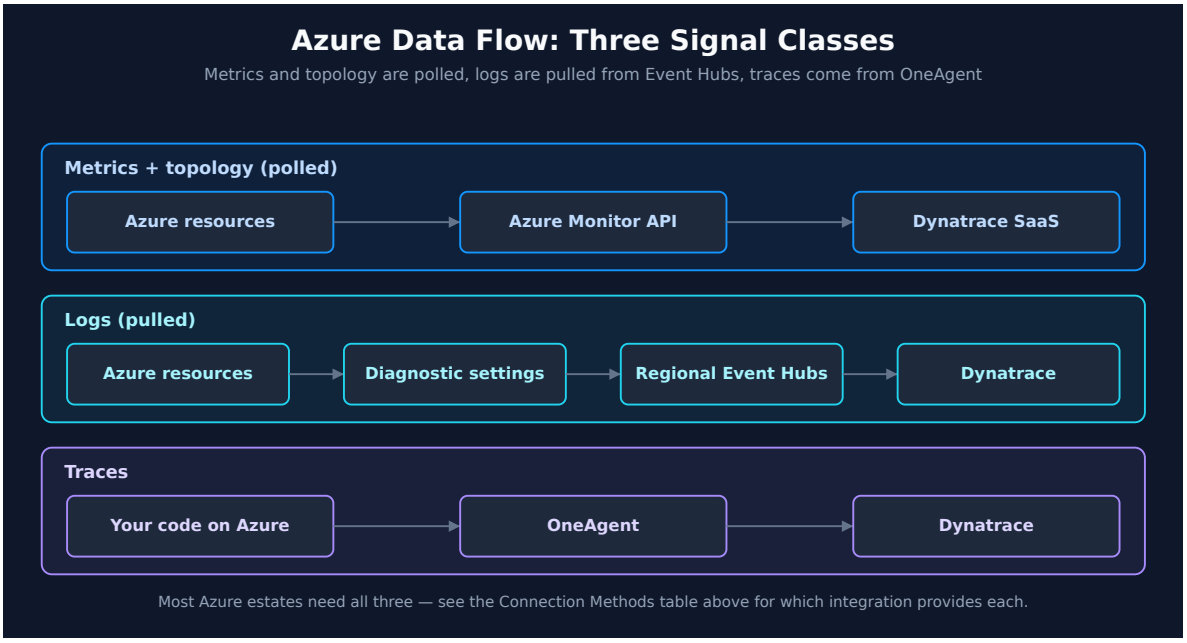
The Azure Native Dynatrace Service is an Azure resource that links an Azure subscription to a Dynatrace environment. Per Microsoft's documentation it provides:

- **Unified billing** — Dynatrace is billed on the Azure invoice

- **Single sign-on** to Dynatrace through Entra ID
- **Metrics collection** — optional at creation; it creates a system-managed identity with the Monitoring Reader role
- **Log forwarding** — subscription activity logs and Azure resource logs, scoped by include/exclude **tag rules**. Diagnostic settings are created on matching resources automatically and removed when a resource stops matching
- **OneAgent management** on VMs, App Service, AKS and Azure Arc machines

Constraints worth knowing before you choose it: *"If there's a conflict between inclusion and exclusion rules, exclusion takes priority."* One set of tag rules applies to every linked subscription, and it adds a diagnostic setting to each matching resource — which counts against Azure's limit of five per resource (§6). Entra ID logs are **not** covered; they need Entra's own tenant-level diagnostic settings.

Data Flow



Clouds (Sovereign) and Regions

The Azure connection covers **Azure public cloud** only. Its *Limitations* list states: *"Azure Government and Azure China sovereign clouds are not supported."* If you operate in a sovereign cloud, confirm the current position with Dynatrace before planning.

Region values are programmatic strings, not display names. Azure regions surface in Dynatrace as lowercase, no-space identifiers — `eastus`, `westeurope`, `southcentralus` — not `East US` / `West Europe`. Filters, tag rules, and segment conditions must match the programmatic form (standardized as of OneAgent 1.337). A condition written against the display name silently matches nothing rather than erroring.

> **Sources:**
>
> - Create an Azure connection (DT docs) — *"Azure Government and Azure China sovereign clouds are not supported."* and *"The new integration does not deploy or use ActiveGate compute resources inside your Azure subscription to poll telemetry"*
> - Azure Native Dynatrace Service (DT docs)
> - Azure monitoring guide — classic (DT docs) — *"At least one ActiveGate needs to be able to connect to Azure Monitor to perform the monitoring tasks."*

> ˜ Manage the Azure Native Dynatrace Service (Microsoft Learn) — *"Diagnostics settings are automatically added to the subscription's resources that match the defined tag rules."*

Release Radar (April 2026): the Clouds app extends to Microsoft Azure

The enhanced **Clouds app** experience — already available for AWS — now extends to **Microsoft Azure**, bringing Azure to parity. (The Release Radar entry says Dynatrace *"extends"* the experience; it does not use the lifecycle term **GA**, so neither does this entry.) For net-new Azure onboarding, this is the recommended path; the connection-mechanism table above still describes what runs underneath.

What the native Azure experience adds:

Capability	What it gives you
Unified resource view	Metrics, logs, metadata, and topology for Azure subscriptions in one place, alongside AWS
Opinionated insights & dashboards	Pre-built dashboards and investigations over enriched Azure telemetry to shorten root-cause analysis
Pre-configured health alerts	Health alerts created and managed directly in the Clouds app, with drill-down, search, and filtering scoped to Azure resources
Broad metric coverage	Any Azure Monitor native platform metric across services, queryable through DQL
Topology inventory	Periodic scanning enriched with native metadata (tags, subscription IDs), fully queryable via DQL
Onboarding & lifecycle management	Centralized provisioning that converts Azure subscriptions into native Dynatrace connections, reducing operational overhead

> **Note:** Azure resource topology and metadata surface through the modern Smartscape model. Each Azure resource type becomes a Smartscape node type named after its ARM type — `microsoft.compute/virtualmachines` becomes `AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES`. §4 uses these node types as the primary inventory query; the classic `dt.entity.azure_*` types can under-count on Clouds-app connections.

2. Authentication Setup

The Azure connection in the Clouds app authenticates with an Entra ID app registration. The Azure Native Dynatrace Service creates its own system-managed identity, so there is nothing to register.

Setup Steps

1. **Create the Azure connection in Dynatrace** — it displays the issuer, subject and audience values the next step needs
2. **Register an application** in Entra ID and add a **federated identity credential** using those issuer, subject and audience values (or, where federation is not possible, a client secret)
3. **Assign the Monitoring Reader role** to the app's service principal at the subscription or management-group scope. When scoping to a management group, use its resource ID rather than its display name

4. **Update the connection in Dynatrace** with the Tenant ID and Client ID (plus the client secret, if you used one)

Required Azure Permissions

Scope	Role	Purpose
Subscription or Management Group	**Monitoring Reader**	The role the Azure connection docs assign to the service principal
Subscription or Management Group	**Reader**	Classic (Settings / ActiveGate) Azure integration

Authentication Methods Comparison

Method	Rotation	Recommended
Federated identity credential (OIDC token exchange; Clouds app connection)	None — no secret exists	**Production (recommended)**
Client secret	Manual; keep expiry < 12 months	Only where federation is not possible
Managed identity	Automatic	Classic ActiveGate running on an Azure VM only

> **Best Practice:** For a Clouds app connection, use a **federated identity credential** — there is no secret to rotate or leak. Managed identity is relevant only if you run the classic integration through an ActiveGate on an Azure VM.
>
> **Sources:** Create an Azure connection via CLI (DT docs) — *"Federated identity credentials provide passwordless authentication and are more secure than client secrets."* The same page assigns the Monitoring Reader role and, for client secrets, notes that Microsoft recommends *"an expiration duration of less than 12 months for enhanced security"*.

3. Supported Azure Services

Dynatrace monitors 50+ Azure services. Key services:

Category	Services
Compute	Virtual Machines, VM Scale Sets, App Service, Azure Functions, Container Instances
Containers	AKS (Azure Kubernetes Service), Container Apps
Database	SQL Database, Cosmos DB, Database for PostgreSQL/MySQL, Cache for Redis
Storage	Blob Storage, File Storage, Queue Storage, Data Lake
Networking	Load Balancer, Application Gateway, Front Door, Traffic Manager, VPN Gateway
Integration	Service Bus, Event Hub, Event Grid, Logic Apps, API Management
AI/ML	Cognitive Services, Machine Learning, OpenAI Service

> **What this table is — and is not.** These are the services covered by **Azure Monitor metrics** ingestion: Dynatrace polls Azure Monitor and stores the resource-level metrics Azure publishes about each service. That is coverage *of the resource*, not of your code's interaction with it. For **code-level visibility** — which application call reached Service Bus, how long a Cosmos DB read took, which span failed and why — you need **OneAgent** on the process making the call (see §1's *OneAgent code-level tracing* row). The two are complementary: Azure Monitor tells you a resource is throttling; OneAgent tells you which service and code path is being throttled. Azure's own service list and Dynatrace's coverage both move; re-check the supported-services list against your tenant's Clouds app rather than treating the table above as exhaustive.

OneAgent Azure SDK Tracing (OneAgent 1.343)

Forthcoming / rolling out with **OneAgent 1.343**, released 07/28/2026. Rollout is **per agent fleet, not per tenant** — **verify the OneAgent version on the hosts concerned**. The tenant version is not the agent version, and fleets commonly lag by more than one sprint.

Azure service	Automatic tracing added for	Note
Service Bus	Java (1.2.31+), Node.js, Python	Covers both sending and receiving messages
Event Hub	Java (1.2.26+), Node.js	Python is **not** part of this release
Cosmos DB	Python	Adds to the runtimes already supported

Azure Functions instrumentation widened in the same release:

Plan / OS	Runtime	Trigger types
Flex Consumption Plan (Linux)	Python	HTTP/webhooks, Service Bus, Event Hubs, Timer, Blob Storage, Event Grid, Azure SQL
Consumption / Premium / Dedicated (Windows)	Java, Node.js	HTTP/webhooks, Service Bus, Event Hubs, Timer

Later OneAgent releases widen Azure Functions coverage further. Each rolls out per agent fleet, so check the OneAgent version on the Function, not the tenant version:

OneAgent	Azure Functions coverage added
1.345 (rollout from 08/25/2026)	Python triggers: Cosmos DB, Kafka, Queue storage, IoT Hub
1.347 (rollout from 09/30/2026)	Python on the **Premium Plan for Linux**
1.349 (rollout from 11/09/2026)	**.NET** on the **Windows Premium and Consumption** plans and the **Linux Premium and Flex Consumption** plans; an updated .NET implementation for the **Dedicated** plan. Java adds the Azure SQL, Cosmos DB and Redis triggers; Python adds the Model Context Protocol and IoT Hub triggers

Until the deployed agent reaches the version that covers a plan and runtime, OpenTelemetry remains the working path for that Function (FAQ-03 walks through the decision). On the Azure SDK side, the same releases add Java Blob Storage clients and Node.js `@azure/cosmos` (1.347), and .NET Event Hub and Service Bus send and receive (1.349).

> **.NET in-process Functions are leaving support.** The end-of-support list names *"OneAgent Feature: Azure Functions .NET In-Process Model"* with last supported OneAgent 1.351. Plan the move to the isolated worker model before the Function's agent goes past 1.351.
>
> **Sources:**
>
> - OneAgent 1.345 release notes (DT docs) — *"Rollout start on Aug 25, 2026"*; *"We extend Python Azure Function support by adding new triggers: Cosmos DB, Kafka, Queue storage, and IoTHub."*
> - OneAgent 1.347 release notes (DT docs) — *"Rollout start on Sep 30, 2026"*; *"Extended instrumentation for Azure Function (tracing, log correlation) for Premium Plan for Linux for Python."*; *"We now support automatic tracing of the Azure SDK Blob Storage Clients in Java applications."*; *"Node.js applications that access Azure CosmosDB via @azure/cosmos package are now traced."*
> - OneAgent 1.349 release notes (DT docs) — *"Rollout start on Nov 09, 2026"*; *"Windows Premium and Consumption plan."*, *"Linux Premium and Flex Consumption plan."*, *"The Dedicated plan has an updated implementation."*; *"We have extended Java Azure Functions support with the Azure SQL trigger."*; *"Java Azure Functions now has extended support with the following triggers: Cosmos DB, RedisPubSubTrigger, RedisListTrigger, and RedisStreamTrigger."*; *"We have extended Python Azure Function support with the following triggers: Model Context Protocol and IoT Hub."*; *"We now support automatic Event Hub and Service Bus tracing (sending and receiving messages) for the Azure SDK for .NET."*
> - End-of-support news (DT docs) — *"OneAgent Feature: Azure Functions .NET In-Process Model"*, under *2026-11-30 End of Support*, *"Last supported OneAgent version: 1.351"*
>
> **Re-baseline Cosmos DB request charges across this upgrade.** OneAgent 1.343 corrected how `db.cosmosdb.request_charge` values are summarized on aggregated spans. Request-charge figures collected **before** 1.343 should be re-baselined rather than compared across the upgrade — a step change at the upgrade boundary is the correction landing, not a workload change.

On OneAgent 1.342 and earlier, the Azure Monitor resource metrics in the table above remain the only coverage for these services — the code-level spans are simply not produced.

Resource Type Mapping

Azure Resource	Smartscape node type (primary)	Classic entity type
Virtual Machine	`AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES`	`dt.entity.azure_vm`
Web App / Function App	`AZURE_MICROSOFT_WEB_SITES`	`dt.entity.azure_web_app`
Cosmos DB account	`AZURE_MICROSOFT_DOCUMENTDB_DATABASEACCOUNTS`	`dt.entity.azure_cosmos_db`
SQL Database	`AZURE_MICROSOFT_SQL_SERVERS_DATABASES`	`dt.entity.azure_sql_database`
Load Balancer	`AZURE_MICROSOFT_NETWORK_LOADBALANCERS`	`dt.entity.azure_load_balancer`
IoT Hub	`AZURE_MICROSOFT_DEVICES_IOTHUBS`	`dt.entity.azure_iot_hub`

Every node type follows the documented naming rule (the ARM resource type, uppercased, with `.` and `/` replaced by `_`), and every row above is confirmed in the semantic dictionary. A node type that exists but has no resources in your subscriptions still returns **zero rows, not an error** — so an empty result tells you nothing about whether you typed the type correctly. Check the dictionary when in doubt:

```
fetch dt.semantic_dictionary.models
| filter data_object == "smartscape.nodes"
| filter startsWith(smartscape_node_type, "AZURE_MICROSOFT_SQL")
| fields smartscape_node_type, name
```

> **Sources:** Azure topology (DT docs). **Dictionary:** `dt.smartscape.azure_microsoft_compute_virtualmachines`, `…_web_sites`, `…_documentdb_databaseaccounts`, `…_sql_servers_databases`, `…_network_loadbalancers`, `…_devices_iothubs`, and classic `dt.entity.azure_vm` / `azure_web_app` / `azure_cosmos_db` / `azure_sql_database` / `azure_load_balancer` / `azure_iot_hub` — all present in `dt.semantic_dictionary.models`, read 09/25/2026.

4. Querying Azure Resources

Azure resources discovered by the Clouds app connection are Smartscape nodes, queried with `smartscapeNodes`. Every Azure node carries `azure.subscription`, `azure.resource.group` and `azure.location`. The classic `dt.entity.azure_*` types still work, but on Clouds-app connections they can under-count or return nothing for the same estate — on the validation tenant (09/24/2026) the classic types saw 1 Azure VM and 0 Web Apps where Smartscape saw 8 and 2.

List Azure Virtual Machines

```
// Azure VMs as Smartscape nodes (primary inventory query)
smartscapeNodes "AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES"
| fields name, azure.subscription, azure.resource.group, azure.location
| sort name asc
| limit 20
```

The classic equivalent, for environments still on the classic integration. `dt.entity.*` is a look-back view: it returns only entities *seen* in the query window, so it needs an explicit `from:`.

```
// Classic: Azure VMs seen in the last 7 days
fetch dt.entity.azure_vm, from:-7d
| fieldsKeep id, entity.name, tags
| sort entity.name asc
| limit 20
```

Count Azure Resources by Type

```
// Count Azure VMs, Web Apps and Cosmos DB accounts (Smartscape)
smartscapeNodes "AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES"
| summarize resource_count = count()
| fieldsAdd resource_type = "Virtual Machine"
| append [
    smartscapeNodes "AZURE_MICROSOFT_WEB_SITES"
    | summarize resource_count = count()
    | fieldsAdd resource_type = "Web App"
  ]
| append [
    smartscapeNodes "AZURE_MICROSOFT_DOCUMENTDB_DATABASEACCOUNTS"
    | summarize resource_count = count()
    | fieldsAdd resource_type = "Cosmos DB account"
  ]
| sort resource_count desc
```

Azure Resources by Resource Group

Resource groups are the unit most Azure teams organize by, so they are the natural first cut for ownership and cost questions.

```
// Azure VMs per subscription and resource group
smartscapeNodes "AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES"
| summarize vm_count = count(), by:{azure.subscription, azure.resource.group}
| sort vm_count desc
| limit 20
```

List Azure Web Apps

```
// App Service and Function App sites (Smartscape); azure.resource.kind distinguishes app /
functionapp / linux
smartscapeNodes "AZURE_MICROSOFT_WEB_SITES"
| fields name, azure.location, azure.resource.group, azure.resource.kind
| sort name asc
| limit 20
```

5. Azure Metrics with DQL

Azure Monitor metrics are ingested into Dynatrace under the `cloud.azure.*` namespace. **Which key you get depends on the connection:**

Connection	Key family	Example
New Azure connection (Clouds app)	`cloud.azure.<namespace>.<MetricName>` — the namespace is the ARM resource type, lowercased with the provider prefix collapsed; the metric name keeps Azure Monitor's casing. Every series carries `dt.da.source == "azure-metric-poller"`	`cloud.azure.microsoft_storage.storageaccounts.Transactions`
Classic connection — built-in services	`builtin:cloud.azure.*` in Metrics Classic (`dt.cloud.azure.*` on Grail)	`builtin:cloud.azure.apiMgmt.capacity`
Classic connection — cloud services	`ext:cloud.azure.*`, all lowercase	`ext:cloud.azure.microsoft_storage.storageaccounts.transactions`

Three things follow when you write or migrate a query:

- **A few keys collide.** A small number of new keys match their classic cloud-service equivalent apart from the `ext:` prefix, and the metric picker shows both under the same display name. Filter on `dt.da.source == "azure-metric-poller"` to isolate the new connection's series. On the validation tenant (11/11/2026) both forms of the Storage key arrived side by side — lowercase `…storageaccounts.transactions` with no `dt.da.source`, and `…storageaccounts.Transactions` from the poller.
- **Dimensions are lowercase on the new connection.** The poller lowercases every dimension name, so query `shardid`, not `ShardId`.
- **Look up each rewrite in the mapping table.** The Azure metric and entity mapping page lists, per resource type, the classic key and the new key for each metric.

The cells below use the new connection's keys. To see which Azure namespaces a tenant is ingesting from the new connection:

```
metrics from:now()-9d
| filter startsWith(metric.key, "cloud.azure.") and dt.da.source == "azure-metric-poller"
| fieldsAdd namespace = concat(splitString(metric.key, ".")[2], ".", splitString(metric.key, ".")
[3])
| summarize keys = countDistinctExact(metric.key), by:{namespace}
| sort keys desc
```

> **Sources:**
>
> - Azure metrics (DT docs) — *"Every metric ingested by this integration uses the same key shape and always carries dt.da.source == "azure-metric-poller""*; *" is the metric name exactly as Azure Monitor defines it, including its casing"*; *"The Azure metric poller lowercases every dimension name before it reaches Grail."*; *"a small number of metrics have a key identical to their classic cloud-service equivalent except for the ext: prefix that classic uses."*
> - Upgrade from classic to new cloud connections (DT docs) — *"Metric keys have two different classic flavours, builtin:cloud.azure.*

> and ext:cloud.azure..“
>
> - Built-in metrics on Grail (DT docs) — the `builtin:` → `dt.` key renaming on Grail
> - Azure metric and entity mapping (DT docs) — Microsoft.Storage/storageAccounts: classic `ext:cloud.azure.microsoft_storage.storageaccounts.transactions` → new `cloud.azure.microsoft_storage.storageaccounts.Transactions`; Microsoft.ApiManagement/service: classic `builtin:cloud.azure.apiMgmt.capacity` → new `cloud.azure.microsoft_apimanagement.service.Capacity`
> - Namespace query executed on the validation tenant 11/11/2026: 8 namespaces from `azure-metric-poller` (`microsoft_compute.virtualmachines` 14 keys, down to `microsoft_network.loadbalancers` 4).

Azure VM CPU Usage

```
// Azure VM CPU (Azure Monitor metric, via the Azure connection) — no OneAgent needed.
//
// Corrected 09/24/2026: this cell used dt.host.cpu.usage with no provider filter, which is the
// OneAgent host metric for EVERY host (EC2, Azure and Kubernetes nodes alike) — not Azure VMs.
timeseries cpu = avg(cloud.azure.microsoft_compute.virtualmachines.PercentageCPU), from:-6h, by:
{azure.resource.name}
| fieldsAdd avgCpuValue = arrayAvg(cpu)
| sort avgCpuValue desc
| limit 10
```

Azure VM Available Memory

```
// Azure VM available memory % (Azure Monitor metric), lowest first, over the last 6 hours.
//
// Corrected 09/24/2026: this cell used dt.host.memory.usage with no provider filter — the OneAgent
// metric for every host, not an Azure VM metric.
timeseries mem = avg(cloud.azure.microsoft_compute.virtualmachines.AvailableMemoryPercentage),
from:-6h, by:{azure.resource.name}
| fieldsAdd avgAvailableMemPct = arrayAvg(mem)
| sort avgAvailableMemPct asc
| limit 10
```

6. Forwarding Azure Logs and Events

Azure does not expose logs through an API that Dynatrace polls. Every Azure log path in this section starts with an Azure Monitor **diagnostic setting** that streams a resource's logs to a destination — for Dynatrace, an **Event Hub**. What differs between the options is who creates the diagnostic settings and what reads the Event Hub.

Three Ways to Get Azure Logs into Grail

Option	Who creates diagnostic settings	What reads the Event Hub	Choose when
Dynatrace pull from Event Hubs (current docs path)	You (ARM template deploys the Event Hubs; you add the diagnostic settings)	Dynatrace SaaS — it discovers tagged Event Hubs namespaces and pulls from them; no function code to host	Net-new log forwarding alongside a Clouds-app Azure connection

Option	Who creates diagnostic settings	What reads the Event Hub	Choose when
Azure Native Dynatrace Service	The service, from include/exclude tag rules	The service	You already use the Native service and want tag-driven, hands-off coverage
`dynatrace-azure-log-forwarder` (open source)	You	An Azure Function App you deploy per region, pushing to the Logs Ingest API with a `logs.ingest` token	Existing deployments; environments that need the push model

What to Forward

Log	Where the diagnostic setting lives	Notes
Activity log (management operations: create, delete, role changes)	Subscription scope	Global — any regional Event Hubs namespace can receive it
Microsoft Entra ID audit logs (sign-ins and directory audit)	Entra tenant scope	Tenant-scoped, not subscription-scoped — needs a tenant-level administrator. Sign-ins have a built-in OpenPipeline bundle, *Azure Entra ID Audit Logs* (FAQ-27 § 2.2)
Resource logs (per-service data-plane logs)	On each resource	The bulk of volume; choose categories deliberately
Events (Event Grid CloudEvents, Azure Monitor alerts)	Event Grid / action groups	Sent to a separate `dt-events-evh` hub, not the logs hub

> **Microsoft 365 audit is not in this table.** Exchange, SharePoint and OneDrive, Teams and DLP audit records live in Microsoft 365's unified audit log and are read from the Office 365 Management Activity API, not from an Azure diagnostic setting, so this route does not deliver them. **FAQ-27** covers that collector and how it pairs with the Entra ID row above: send Entra ID sign-ins through this route, and leave `Audit.AzureActiveDirectory` out of the collector so they are not collected twice. The *Microsoft 365, Office 365* Hub extension is not a substitute — it produces service-health and usage metrics, and *"The extension uses audit logs to count O365 service users."*
>
> **Sources:** Microsoft 365, Office 365 extension (DT docs), Management Activity API reference (Microsoft Learn).
>
> **Logs that land only in a storage account.** The Dynatrace Azure logs route reads Event Hubs, so a source that Azure writes only to a storage account is outside it. Virtual network flow logs are the common case: they are a Network Watcher feature rather than a diagnostic setting, and *"Flow data from virtual network flow logs is sent to Azure Storage."* For such a source, the `azureblobpolling` receiver for the OpenTelemetry Collector can poll the container and forward the records to Dynatrace. It complements the Event Hubs route for sources that route cannot reach; it does not replace it. The receiver is maintained in the `dynatrace-bindplane-otel-contrib` repository, its stability is **beta**, and it ships only in the full Dynatrace Bindplane OTel Collector build — not the slim build and not the Dynatrace OTel Collector, whose support *"covers only the components and their versions that are included in the Dynatrace OTel Collector."* Bindplane can manage it but is not required. Read at v1.2.0:

- **Set `enable_incremental_read: true`.** It is off by default, and the default path reads each blob once, whole. Flow logs are *"ingested into a block blob at one-minute intervals by appending blocks"*, so records appended after that first read are never ingested. On the default path a failed blob read is only logged and the poll window still advances, so that blob is not retried. The README calls the flag temporary: it is meant to become the default.
- **Use `blob_format: records-json`** for Azure's `{"records":[...]}` exports, so each array element becomes its own log record.
- **Connection string only.** The receiver authenticates with the storage account's connection string; there is no managed-identity option.
- **Event time is not set.** Records carry only the observed (collection) timestamp. If you need when the event happened, parse the record's `time` field, for example in OpenPipeline.
- **One collector per container or prefix.** Progress is checkpointed in the collector's own `storage` extension and is not shared between collectors, so two instances reading the same blobs ingest them twice. Without a `storage` extension, a restart falls back to `initial_lookback`.

Sources:

- Forward Azure logs (DT docs) — *"This page explains how to forward Azure logs to Dynatrace via Event Hubs."*
- Virtual network flow logs (Microsoft Learn) — *"Flow data from virtual network flow logs is sent to Azure Storage."*
- OpenTelemetry Collector (DT docs) — *"This support covers only the components and their versions that are included in the Dynatrace OTel Collector."*
- azureblobpollingreceiver README (Dynatrace GitHub) — *"When it is off (the default), every format keeps the legacy behavior of parsing each blob whole on every poll and deduping by name, which loses records appended to a blob after it is first read."*
- azureblobpollingreceiver metadata (Dynatrace GitHub) — *"beta: [metrics, logs, traces]"*
- DBDOT Collector manifest (Dynatrace GitHub) — *"receiver/azureblobpollingreceiver v1.2.0"*
- **Code read (10/29/2026):** at tag `v1.2.0`, `config.go` leaves `enable_incremental_read` false by default (`factory.go` does not set it) and requires `connection_string`; `internal/azureblob/blob_client.go` builds its client only with `NewClientFromConnectionString`; on the default path a blob read error is logged ("Error consuming blob", `receiver.go`) and `finalizePoll` still advances the poll time; the `internal/blobconsume` consumers set `ObservedTimestamp` and never `Timestamp`; the checkpoint is one key per receiver instance in its storage extension. Not run against an Azure storage account
- **Catalog observation (10/29/2026):** no `azureblobpollingreceiver` entry in the slim DBDOT manifest or the Dynatrace OTel Collector manifest

Setting Up the Dynatrace Pull Path

1. **Deploy the ARM template** from the Azure logs page into each Azure region that hosts resources you want logs from. It creates an Event Hubs namespace with a `dt-logs-evh` hub (logs) and a `dt-events-evh` hub (events).
2. **Leave the namespace tags in place.** Dynatrace only connects to namespaces carrying its `managed-by` and `dt-log-ingest-activated` tags, and a namespace serves one Azure connection.
3. **Make sure the Dynatrace service principal holds *Azure Event Hubs Data Receiver*** on every regional deployment — the Azure logs page describes which assignments the template makes and which an administrator must add per region.
4. **Create diagnostic settings** that stream to the regional namespace — activity log at the subscription, Entra ID at the tenant, resource logs on each resource.

Diagnostic Settings Rules That Shape the Design

Rule	Consequence
"Each resource can have up to five diagnostic settings."	Existing SIEM, Log Analytics and Native-service settings all compete for the five slots. Inventory them before adding one.
"For regional resources, the destination must be in the same region as the monitored resource (applies to Storage accounts and Event Hubs)."	One Event Hubs namespace **per region** — a single central hub cannot receive resource logs from other regions.
Category groups: `allLogs` or `audit`	Once you choose a category group you cannot also pick individual categories in that setting.
"Diagnostic settings don't allow granular filtering within a selected category."	Volume control happens by category choice at the source, then by OpenPipeline drop/filter rules in Dynatrace.
Data can take up to 90 minutes to start flowing after a setting is created	Do not conclude a setting is broken from an empty result in the first hour.

> **Azure-side cost.** Event Hubs throughput units and, for some categories, the export itself are billed by Azure — independently of Dynatrace ingest. Check the *Costs to export* column for a category before enabling `allLogs` on high-volume services.

Find Azure Logs in Grail

Each forwarding path marks its records differently, and the documented marker is the only reliable filter:

Path	Documented filter	Also written
Dynatrace pull from Event Hubs	`dt.da.source == "azure-log-ingest"`	—
`dynatrace-azure-log-forwarder`	`dt.openpipeline.source == "/api/v2/logs/ingest"` and `cloud.provider = "Azure"`	`cloud.log_forwarder`, `azure.resource.id`, `azure.resource.type`, `azure.resource.group`, `azure.subscription`; `log.source` such as `Activity Log - Administrative`

> **Do not filter on `cloud.provider == "azure"` alone.** OneAgent stamps `cloud.provider` and `azure.resource.id` on every log it ships from an Azure host — including every AKS node. On a live tenant on 09/25/2026 that filter matched **206,565** records in 24 hours, **all** of them OneAgent host and container logs, and **not one** Azure resource or activity log. A dashboard built on it reports healthy Azure log ingest when none exists.

```
// Azure platform logs (activity, Entra ID, resource logs) by path, source and resource type.
// The two path markers are the documented ones. Do NOT use `cloud.provider == "azure"` alone:
// OneAgent stamps it on every host and container log from Azure VMs and AKS nodes.
fetch logs, from:-1h
| filter dt.da.source == "azure-log-ingest"
    or (dt.openpipeline.source == "/api/v2/logs/ingest" and lower(cloud.provider) == "azure")
| summarize log_count = count(), by:{dt.da.source, log.source, azure.resource.type}
| sort log_count desc
| limit 20
```

If this returns nothing, work through the causes in order before concluding that no logs arrived: diagnostic settings created less than 90 minutes ago; an Event Hubs namespace in a different region from the resource; a missing *Event Hubs Data Receiver* role; the query window (widen to `from:-24h`); and finally the marker — inspect one record from your log source with `fetch logs, from:-1h | filter contains(content, "<your resource name>") | limit 1` and read its `dt.da.source`, `dt.openpipeline.source` and `cloud.log_forwarder` values.

AKS Control-Plane Logs

AKS control-plane components (`kube-apiserver`, `kube-audit`, `kube-audit-admin`, `kube-controller-manager`, `kube-scheduler`, `cluster-autoscaler`, `guard`, and others) are exposed **only** as resource logs of the AKS cluster resource, so they reach Dynatrace through this same diagnostic-settings path. Microsoft's cost warning applies directly:

> *"You can incur substantial cost when you collect resource logs for AKS, particularly for kube-audit logs."* Microsoft recommends enabling `kube-audit-admin`, *"which excludes the `get` and `list` audit events."*
>
> **Sources:**
>
> - Azure logs and events (DT docs)
> - Forward Azure logs (DT docs) — *"Use the dt.da.source attribute to filter for logs ingested through the Azure logs ingest pipeline."*
> - Set up the Azure log forwarder (DT docs) — *"If you already have multiple integrations, you can additionally use the values cloud.log_forwarder and dt.auth.origin to further refine your filters."*
> - dynatrace-azure-log-forwarder (Dynatrace GitHub)
> - **Dictionary:** `dt.da.source` (`experimental`), `dt.openpipeline.source` (`experimental`), `log.source` (`stable`), `cloud.provider` (`stable`), `azure.resource.id` (`experimental`), read 09/25/2026.
> - Diagnostic settings in Azure Monitor (Microsoft Learn) — *"Each resource can have up to five diagnostic settings."*
> - Monitor AKS (Microsoft Learn) — *"You can incur substantial cost when you collect resource logs for AKS, particularly for kube-audit logs."*

7. AKS Monitoring

Azure Kubernetes Service (AKS) runs the same Dynatrace Operator and DynaKube custom resource as EKS (CLOUD-03) and GKE (CLOUD-06). What is Azure-specific is how you install it, which node types it can cover, and which AKS guardrails it has to pass.

Installing the Operator on AKS

Option	What it is	Trade-off
AKS Marketplace extension	Installs Dynatrace Operator and a DynaKube as an AKS cluster extension (`az k8s-extension create … --cluster-type managedClusters`)	Quickest; the extension's DynaKube is not fully customizable (for example, extra tolerations)
Helm chart + DynaKube manifest	The standard Operator install	Full control of the DynaKube; fits GitOps (K8S series)
Azure Native Dynatrace Service	OneAgent management for AKS from the Azure portal	Convenient when the Native service is already your integration

Write new DynaKube manifests against `dynatrace.com/v1beta6`. The Operator CRD marks `v1beta5` as deprecated: *"This dynakube API version is deprecated and will be removed in a future operator version."*

What Can Be Monitored Where

AKS node type	Dynatrace coverage
Linux node pools — Ubuntu or Azure Linux	Full coverage (`cloudNativeFullStack` , `applicationMonitoring` , `hostMonitoring`); Azure Linux is a supported container host
Windows Server node pools	Not supported: Dynatrace's technology-support matrix lists AKS with the footnote *"Windows Pods and Nodes unsupported."* The one documented exception is manual **pod runtime injection** of the .NET code module into Windows containers — application-only, outside the Operator, and it *"may not fully link Kubernetes workloads with monitored containers/processes"*
Virtual nodes (ACI)	No host agent: Microsoft states *"DaemonSets won't deploy pods to the virtual nodes."* Host-level and cloud-native full-stack coverage cannot reach pods scheduled there

AKS Guardrails That Affect the Operator

AKS Automatic and Deployment Safeguards. On AKS Automatic, *"Deployment Safeguards and baseline Pod Security Standards are enabled by default in `Enforce` mode. You can exclude namespaces, but you can't switch the cluster-wide safeguard level to `Warn` ."* The baseline standard rejects host namespaces (`hostNetwork` , `hostPID`), privileged containers and added capabilities — and the Operator's OneAgent DaemonSet runs with `HostNetwork` , `HostPID` and capabilities such as `SYS_ADMIN` . Exclude the `dynatrace` namespace from Deployment Safeguards before you install, and verify that injected application pods are admitted in a non-production cluster first.

Network. The Operator's webhook listens on TCP 8443, and the Kubernetes API server must be able to reach it for injection to work — check this path on private clusters and clusters with restrictive network policies. Operator components also need outbound HTTPS (443) to your Dynatrace environment.

Kubenet retirement. Kubenet (legacy) *"Retires on March 31, 2028"* — Microsoft directs migration to Azure CNI Overlay. A node-pool network migration recreates nodes, so plan it alongside an Operator upgrade window rather than as a separate outage.

AKS vs EKS at a Glance

Aspect	AKS	EKS
Operator install	AKS Marketplace extension or Helm	Helm (see CLOUD-03)
Windows nodes	Unsupported (technology-support matrix)	Not stated on the technology-support matrix — verify for your Operator version
Serverless pods	Virtual nodes (ACI) — no DaemonSet	Fargate — no DaemonSet
Policy guardrails	Deployment Safeguards (enforced on AKS Automatic)	Pod Security admission (configurable)
Control-plane logs	Resource logs via diagnostic settings (§6)	CloudWatch control-plane logging (CLOUD-07)

> **Control-plane metrics stay in Azure.** Microsoft states *"The control plane metrics feature supports only the managed service for Prometheus in Azure Monitor."* Dynatrace's in-cluster data comes from the Kubernetes API (via the ActiveGate `kubernetes-monitoring` capability) and the node agents; for API-server behaviour inside Dynatrace, forward the control-plane resource logs described in §6.

Namespace Resource Usage

These queries use the Kubernetes metrics the Operator produces. Replace `<your-aks-cluster>` with your cluster's name to scope them to one AKS cluster — without the filter they span every monitored cluster, on any cloud.

```
// Namespace CPU on one AKS cluster: SUM of container CPU per namespace
// (avg would report the per-container mean, not what the namespace consumes)
// rollup: avg averages each container WITHIN a time bucket before summing across containers;
// without it, widening the timeframe to 24h (10-min buckets) inflates the result ~10x.
timeseries nsCpu = sum(dt.kubernetes.container.cpu_usage, rollup: avg), from:-1h,
    by:{k8s.namespace.name},
    filter:{k8s.cluster.name == "&lt;your-aks-cluster&gt;"}
| fieldsAdd avgCpu = arrayAvg(nsCpu)
| sort avgCpu desc
| limit 10
```

```
// Namespace memory (working set) on one AKS cluster, in MiB
timeseries nsMem = sum(dt.kubernetes.container.memory_working_set, rollup: avg), from:-1h,
    by:{k8s.namespace.name},
    filter:{k8s.cluster.name == "&lt;your-aks-cluster&gt;"}
| fieldsAdd avgMemMiB = arrayAvg(nsMem) / 1048576.0
| sort avgMemMiB desc
| limit 10
```

Sources:

- Dynatrace Operator on the AKS Marketplace (DT docs)
- Technology support — Microsoft Azure (DT docs) — *"Windows Pods and Nodes unsupported."*
- Pod runtime injection (DT docs) — *"This option refers to .NET applications in Windows containers."*

- Kubernetes supported distributions (DT docs)
- Azure Linux container host support (Dynatrace blog)
- DynaKube CRD (Dynatrace GitHub) — *"This dynakube API version is deprecated and will be removed in a future operator version."*
- Operator network requirements (Dynatrace GitHub)
- Virtual nodes (Microsoft Learn) — *"DaemonSets won't deploy pods to the virtual nodes."*
- Deployment Safeguards (Microsoft Learn) — *"Host network namespaces are disallowed"*
- Dynatrace Operator security (DT docs) — OneAgent DaemonSet policy settings: *"Allows HostNetwork"*, HostPID, and capabilities including `SYS_ADMIN`
- CNI overview (Microsoft Learn) — *"Retires on March 31, 2028"*
- Control plane metrics (Microsoft Learn) — *"The control plane metrics feature supports only the managed service for Prometheus in Azure Monitor."*
- **Derived:** excluding the `dynatrace` namespace follows from the baseline host-namespace and capability rules (Deployment Safeguards) combined with the OneAgent DaemonSet's policy settings (Operator security).

8. Subscription and Resource Group Governance

Azure organizes resources as management groups → subscriptions → resource groups, with tags across all of them. Dynatrace carries this structure as fields on the data (`azure.subscription`, `azure.resource.group`, `azure.location`, plus your Azure tags), so governance is a matter of using those fields in the right Dynatrace construct.

Mapping Strategy

Question	Dynatrace construct	Azure field it keys on
Who may see this data?	IAM policies (with bucket permissions or a security context)	Subscription or resource group
What does this team look at by default?	Segments	Resource group, subscription, tags
Who gets paged?	Problem-triggered workflows	Ownership tags (`owner`), resource group
Who pays?	Cost allocation fields	Subscription, `cost-center` tag

> **Segments filter; they do not restrict access.** *"Regardless of configured visibility, any segment can be accessed with storage:filter-segments:read permission."* A segment per resource group is a good default *view*; it is not an access boundary. Management Zones and alerting profiles are the classic equivalents of segments/IAM and workflows respectively.

Best Practices

- **Enforce Azure tags at the source** (Azure Policy) — `environment`, `team` / `owner`, `cost-center` — so they arrive on every resource and log

- **Build segments on** `azure.resource.group` **and** `azure.subscription` for scoped views
- **Grant access with IAM policies**, not segments; keep regulated subscriptions' logs in their own bucket if access must be enforced at storage
- **Route problems with workflows** keyed on ownership tags, not on resource-group names alone — resource groups are renamed less often than teams change, but ownership is what routing needs
- **Use naming conventions** that encode application, environment and region: `rg-myapp-prod-eastus`

> **Sources:** Visibility of segments (DT docs) — *"Regardless of configured visibility, any segment can be accessed with storage:filter-segments:read permission."*

Tagging this producer — primary Grail tags

Azure follows the AWS pattern in CLOUD-02: an **Azure tag** rule at subscription scope forwards the tag as `azure.tags.<key>` (or writes `dt.security_context` / cost fields directly), and an OpenPipeline primary Grail tag rule promotes it with two sources — `azure.tags.<key>` for telemetry and `` `tags:azure`[<key>] `` for Azure Smartscape nodes — under an `isNull(primary_tags.<key>)` condition. `azure.subscription` and `azure.resource.group` are set automatically and need no rule.

- **OneAgent on Azure VMs** needs a host-group **Host/Process property** rule (OneAgent 1.343+), the same as EC2: *"Azure VMs follow the same pattern."* — with `azure.tags.<key>` as source.
- **Azure Functions** take `DT_TAGS`, like Lambda (CLOUD-04).
- **Availability.** The Azure page no longer describes central configuration as expected; it now lists it as the recommended option: *"Promote Azure tags with central configuration"*. Confirm the *Azure tag* rule type exists in your tenant's Ingest enrichment configuration before relying on it.

> **OneAgent 1.349 (rollout from 11/09/2026) — breaking change for the Host/Process property route.** The OneAgent 1.349 release notes (DT docs) say: *"In the Host/process property rules dropdown, the aws.tags and azure.tags based rules will temporarily no longer work as sources or in conditions for injected processes. Full cloud tags support, which brings that functionality back, is planned for a future release."* OneAgent updates roll out over weeks, so check the version your hosts run before relying on this route for data from injected processes; on OneAgent versions before 1.349 the route described above remains the working path.
>
> **Sources:** Enrich Azure telemetry with primary Grail fields and tags (DT docs) — *"Promote Azure tags with central configuration"*, Apply primary Grail tags across an example AWS estate (DT docs) — *"Azure VMs follow the same pattern."*

9. Summary and Next Steps

Key Takeaways

- **Onboard with an Azure connection in the Clouds app**: Entra ID app registration, **Monitoring Reader**, and a **federated identity credential** — no secret to rotate, no ActiveGate to run
- **The Azure Native Dynatrace Service** trades control for convenience: Azure billing, SSO, tag-rule-driven log forwarding and OneAgent management
- **Query Azure resources as Smartscape nodes** (`AZURE_MICROSOFT_*`); the classic `dt.entity.azure_*` types can under-count on Clouds-app connections

- **Logs travel through diagnostic settings to regional Event Hubs** — one namespace per region, five settings per resource, and category choice is your first volume control
- **AKS** runs the standard Operator (`v1beta6` DynaKube); Windows node pools and virtual nodes are outside host-level coverage, and AKS Automatic needs the `dynatrace` namespace excluded from Deployment Safeguards
- **Govern with IAM policies for access and segments for views**, keyed on `azure.subscription` and `azure.resource.group`

Next Steps

- **CLOUD-06: GCP Integration** — Google Cloud monitoring setup
- **CLOUD-07: CloudWatch Log Ingestion** — the AWS counterpart to §6
- **CLOUD-08: Multi-Cloud Patterns** — unified monitoring across Azure and other providers
- **K8S series** — DynaKube configuration in depth

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.